

Architecture & Verification Dossier

The complete technical account of the platform's two named layers — **XELOR**, the system of record, and **ONYX**, the intelligence layer above it — written so that every structural claim in it can be re-proved from the repository by a reviewer who trusts none of it.

Purpose: architecture review and independent verification. This supersedes the *MVP Technical Handoff Brief*, which stopped at the Phase 1 investor build.

Audience: senior/principal architects, engineering leadership, technical due diligence, incoming platform engineers.

Method: every number in this document was measured against the running system and the checked-out source on the date below. Commands are given so each can be re-run.

Scope: **XELOR** = `PHASE-1`, the ERP baseline · **ONYX** = `PHASE-2`, autonomous fulfilment and the AI module. PostgreSQL 17.10, 96 migrations, 995 automated tests.

Measured from the live stack, the schema, the route table, the CI gates and the test runners — not from design documents

16 August
2026

0. The verification contract

This document makes claims about a system. A claim that cannot be checked is an opinion, so every substantive section carries the command that re-derives it. If a command's output disagrees with the text, the text is wrong.

How to reproduce every number in this document in one pass. From [PHASE-2/](#), with the infrastructure up:

```
pnpm build && pnpm lint && pnpm typecheck && pnpm db:migrate && pnpm test \
  && pnpm ai:eval && pnpm db:rls-check && pnpm db:perm-check \
  && pnpm db:naming-check && pnpm module-check && pnpm report-data-check
# this exact chain is the `pnpm verify` script and job 1 of .github/workflows/ci.yml
```

What is being asserted, and at what strength

Strength	Meaning	Example of a claim at this strength
Enforced	A machine rejects the change. CI fails, a constraint fires, or a lint rule blocks the merge.	"No module imports another module's internals" — <code>eslint-plugin-boundaries</code> , CI job 1.
Measured	Observed on the running system on the date of this document. Re-running may give a different number as data grows.	"List endpoints answer in 10–22 ms" — at 6,208 live rows.
Implemented	The code path exists and executes, but nothing prevents a future change from removing it.	"The mission never writes stock directly."
Stated	A design intent recorded in the binding decisions document, not yet mechanically enforced.	"RLS overhead above 15–20% triggers a design review."

Read §18 before forming a view. The gap and risk register is not an appendix of nice-to-haves. It contains the four things a reviewer would otherwise find themselves, including one schema inconsistency, one demo scenario that cannot start, and the precise blast radius of the public-demo authentication bypass.

Document map

- 1

System in one page
- 2

Codebase topology and boundary enforcement
- 3

XELOR — the system of record
- 4

Data architecture
- 5

Multi-tenancy and row-level security
- 6

Transaction, consistency and eventing model
- 7

Document numbering and statutory constraints
- 8

API contract
- 9

Identity and authorisation
- 10

Audit, evidence and tamper-evidence
- 11

ONYX — the intelligence layer
- 12

The mission arc
- 13

Determinism and AI governance
- 14

The AI module — providers, models, governance
- 15

Failure, retry and recovery
- 16

Verification evidence
- 17

Build, run and deploy
- 18

Gap, risk and decision register
- 19

The reviewer's checklist

1. System in one page

XELOR is a boundary-enforced modular monolith: one deployable API, one deployable web application, one PostgreSQL database with forced row-level security, and a set of internal modules that are prevented — mechanically, in CI — from reaching into one another.

153,961

lines of TypeScript across 648 source files

252

tables · 242 tenant-scoped, all FORCE RLS

449

HTTP routes · 165 distinct permissions

995

automated tests · 994 pass, 1 environment-skipped

The two layers, and why they are two

The platform is exactly two layers, and they are named. **XELOR** is a conventional ERP: the system of record, and authoritative. **ONYX** is the intelligence and automation layer that sits *on top* of XELOR and reaches it only through declared ports. The separation is architectural, not presentational — ONYX contains no import of any XELOR module.

One name, two scopes — read this before §11 and §14. **ONYX** names the whole upper layer in this document. Inside the codebase the same four letters also name (a) the AI Operations component that owns the provider router and feature registry (§14), and (b) the coordinating agent in the mission arc, as distinct from AXLE, SPAR, KILN and HEXA (§12). All three usages are the same idea at different zoom levels — the intelligence of the product — but a reviewer reading `agentKey: "ONYX"` on a mission step is looking at the narrowest of the three.

XELOR — system of record (Phase 1)		ONYX — intelligence layer (Phase 2)
Answers	"What is true, and what did we commit to?"	"Given what is true, what should we do, and did it work?"
Owns	Every table. All writes. All numbering. All statutory records.	Missions, plans, approvals, actions and evidence. Nothing in a XELOR domain.
Reaches the other by	— XELOR does not know ONYX exists. No XELOR module imports, references or depends on anything in the ONYX tree.	15 injected port tokens; never a module import.
Code footprint	202 API files · 122 platform files · 90 migrations	+21 API files · +19 platform files · +6 migrations (0092–0097)
Added directories	—	<code>apps/api/src/fulfilment/ · packages/platform/src/{fulfilment,intelligence,data import}/</code>
Runs on	<code>api :3000 · web :3001 · db indcore</code>	<code>api :3100 · web :3101 · db indcore_p2</code>

Why the two layers get separate databases and separate ports. ONYX runs six migrations XELOR has never seen. A shared database would mean that starting ONYX once silently rewrote XELOR into a state that no longer matches its own code — and the XELOR build is the baseline against which ONYX's value is demonstrated. The separation is enforced by `run-phase.sh`, which frees the other layer's ports before binding its own.

Runtime topology



Component	Image / runtime	Responsibility, and what fails if it is absent
PostgreSQL	<code>pgvector/pgvector:pg17</code>	Every durable fact. Extensions in use: <code>vector</code> , <code>pg_trgm</code> , <code>btree_gist</code> , <code>plpgsql</code> . Absent → nothing starts; migration 0000 requires all three non-default extensions.
API	<code>Node 22 · NestJS 11</code>	All domain logic, all authorisation, all transaction boundaries. The web app holds no business rule it can act on alone.
Web	<code>Next.js 15 · React 19</code>	24 registered modules. Proxies <code>/api/v1</code> to <code>NEXT_PUBLIC_API_ORIGIN</code> ; misconfiguring this points ONYX's UI at XELOR's API, which answers 401 rather than failing loudly.
Keycloak	<code>keycloak:26.0</code>	OIDC identity. Absent → sign-in fails; public-demo mode still works, which is why 12 e2e tests skip rather than fail in that configuration.
Valkey	<code>valkey/valkey:8</code>	Cache and BullMQ queues. Absent → background work stalls; synchronous ledger paths are unaffected by design (§6).
Gotenberg	<code>gotenberg:8</code>	Server-side PDF rendering for documents produced by the product. Distinct from the local report pipeline that produced this file.

Edge worker	apps/edge	Small runtime for serverless invocation paths. 6 tests.
-------------	-----------	---

VERIFY §1

```
docker compose -f infra/docker-compose.yml config --services
psql "$DATABASE_URL" -c "select extname from pg_extension order by 1"
psql "$DATABASE_URL" -c "select version()"
PostgreSQL 17.10 · btree_gist, pg_trgm, plpgsql, vector
```

2. Codebase topology and boundary enforcement

The single most load-bearing architectural decision in this codebase is that module boundaries are enforced by a linter that fails the build, not by a convention in a README. Everything else — the ports, the outbox, the phase separation — depends on that rule actually biting.

Workspace layout

Package	Files	Lines	May depend on
packages/platform	141	31,676	Nothing internal. Pure logic: errors, ids, tenancy, audit, events, access, tax, accounting, planning, quality, fulfilment, intelligence. This is why 822 of the 995 tests live here — it is testable without a database.
packages/db	42	7,895	platform only. Drizzle schema, 96 migrations, and five executable proof scripts.
apps/api/src/modules/*	—	—	platform + db + its own module only . Cross-module access goes through a sibling's public <code>index.ts</code> or an outbox event.
apps/api (rest)	223 total	55,512	Bootstrap, common middleware, ports, agent-os, fulfilment.
apps/web	242	58,878	24 modules under a shared spine. Module manifests are validated in CI.
apps/edge	3	—	Serverless runtime shim.

The rule, as the linter states it

From `eslint.config.js` — the element model and the load-bearing constraint:

```
// platform -> packages/platform (the bootstrap; may depend on nothing internal)
// db       -> packages/db      (schema/migrations; may depend on platform only)
// module   -> apps/api/src/modules/<name> (one ERP domain each)
// app      -> apps/api/src (everything else: bootstrap, common)
// The load-bearing rule: a `module` may import platform + db + ITS OWN module only.
// Cross-module access must go through a sibling module's public `index.ts`
// (exported service interface) or an outbox event — never a deep import.
```

A subtlety worth checking in review. The config carries an explicit note that a TypeScript-aware resolver is *required* for these rules to bite — without it the plugin cannot resolve `.js`→`.ts` imports and "silently classifies nothing", i.e. the gate passes while enforcing nothing. The resolver is configured against every `apps/*/tsconfig.json` and `packages/*/tsconfig.json`. A reviewer should confirm this survives any future build-tool change, because its failure mode is silent.

What the boundary buys, concretely

Two named doctypes in this schema would collapse into one under a weaker boundary. `production_order` (a shop-floor job) and `maintenance_work_order` (an asset repair) are distinct records owned by distinct modules, and the naming proof asserts there is **no foreign key between them** — the check exists because the two are colloquially both "work orders" and merging them is the obvious wrong refactor.

VERIFY §2 — measured 16 Aug 2026

```
pnpm lint          # eslint . --max-warnings=0, includes boundary rules
pnpm module-check
Module check OK — 24 module(s) installed and registered; 169 nav permission(s)
all defined by the platform; no cross-module imports.
pnpm db:naming-check
Naming check OK — 252 tables; production_order and maintenance_work_order
are distinct doctypes with no FK between them.
```

3. XELOR — the system of record (Phase 1)

XELOR is a complete manufacturing ERP and it is the authority for every durable fact in the product. ONYX has no table of its own in any XELOR domain and no privileged path into one. Everything in this section exists independently of, and before, the intelligence layer — XELOR runs, and is useful, with ONYX switched off entirely.

3.1 Ownership model — six departments, one cross-cutting component

Every module has exactly one owning department. The naming system is deliberate: four letters, industrial register, and the departmental initial matches the owner's initial. These are internal engineering identifiers, not customer-facing branding.

Agent	Department	Owns — system of record
HEXA	Platform & Governance	Master data, identity, RBAC/ABAC, the workflow engine, the hash-chained audit trail, the outbox event bus, external connectors. Owns the binding decisions document itself.
MICA	Commercial	Customer master, leads, quotations, sales orders, tenders, dispatch and invoicing; service tickets, complaints, warranty and AMC.
SPAR	Supply Chain	Vendor master, the PO→GRN→invoice→payment cycle, and the stock ledger itself — bins, valuation, batches, and the single write path every other module must use.
AXLE	Product Engineering & Planning	Item, BOM and routing masters, ECR→ECO change control, MPS, MRP, capacity and finite scheduling.
KILN	Manufacturing Operations	Work orders, material issue and output, scrap, batch genealogy, quality gates, NCR and CAPA, calibration, asset uptime and downtime.
RASP	People & Money	Employee master, shifts and rosters, attendance, leave, payroll and Indian statutory compliance, budgets, expense claims, indirect spend, the general ledger.
ONYX	AI Operations (component, not a department)	Provider-agnostic router, closed feature registry, prompt lifecycle, golden-set eval gates, guardrails, cost ledger, PII egress control, human-in-the-loop queue, kill switch.

A named hazard, accepted knowingly. SPAR and RASP are anagrams. The mitigation recorded in the governance document is that the owner's initial disambiguates them and neither may ever be abbreviated in code, package names, branches or tickets. A reviewer should treat any three-letter form of either as a defect.

3.2 The module map

21 API modules serve 24 registered web modules. Route counts are the count of `@Get/@Post/@Patch` decorators in each module's controllers.

Module	Owner	Route prefix	Routes	Tables	Substance
maintenance	KILN	<code>/maintenance</code>	53	23	Assets, preventive schedules, breakdown requests, downtime, spares consumption, calibration.
csp	MICA	<code>/portal</code>	41	22	Customer service portal: tickets, SLA clocks, complaints, warranty, AMC, spares orders.
expenditure	RASP	<code>/expenditure</code>	40	23	Budgets, indirect spend, expense claims and their approval policy.
administration	HEXA	<code>/admin</code>	36	19	Roles, permissions, segregation of duties, audit browsing, tenant configuration.
hrm	RASP	<code>/hrm</code>	35	27	Employees, shifts, rosters, muster, attendance, leave, payroll and Indian statutory outputs.
aiops	ONYX	<code>/aiops</code>	33	17	AI router, feature registry, prompt versions, eval runs, cost ledger, kill switch, connectors.
planning	AXLE	<code>/planning</code>	33	21	MPS, MRP runs, planned orders, pegging, capacity load, plan exceptions, policies.
integration	HEXA	<code>/integration</code>	32	16	External connectors, flows, dead-letter handling, replay.
quality	KILN	<code>/quality</code>	17	9	Inspection plans, results, sampling, NCR, CAPA, quality releases.
purchase	SPAR	<code>/purchase/*</code>	15	5	Vendor master, purchase orders and lines, GRN, vendor invoice matching.
sales	MICA	<code>/sales</code>	12	5	Customer master, sales orders and lines, credit gate, dispatch, GST invoicing.
production	KILN	<code>/production/orders</code>	11	3	Work orders, component issue, output confirmation, scrap.
engineering	AXLE	<code>/engineering/boms</code>	8	3	Items, BOMs and BOM lines. One active revision per item, enforced by migration 0092.
accounts	RASP	<code>/accounts</code>	8	7	Chart of accounts, journal vouchers, trial balance, customer outstanding.

dataimport	HEXA	/dataimport	6	2	Spreadsheet/CSV onboarding: upload, sheet selection, column mapping, per-row validation and commit.
inventory	SPAR	/stock/entries	5	5	Stock balances, bins, batches, valuation — and the single write path (§3.3).
workflow	HEXA	/workflow	5	3	The <code>WorkflowExecutor</code> port and its instances/actions.
copilot	ONYX	/copilot	4	1	Grounded question answering over the tenant's own records.
platform-health	HEXA	/platform-health	3	1	ACHILES self-assurance checks and their history.
general	HEXA	/general/companies	2	2	Company, plant and department masters.
managed-services	—	/managed-services	1	—	RELAY managed-service lifecycle presentation.

Web-only modules with no distinct API prefix: `agentos`, `aicontrol`, `fulfilment` (Phase 2), `working-capital`. Registry: `apps/web/src/modules/registry.ts`.

3.3 The single write path — the rule that keeps the ledger honest

Stock is the most contended resource in a manufacturing ERP and the easiest to corrupt: five modules have a legitimate reason to move it. XELOR resolves this by giving Inventory sole ownership of the stock tables and forcing every other module through one endpoint.

```
// apps/api/src/modules/inventory/stock.controller.ts
* THE single write path to stock (DECISIONS-V2 §5.6): POST /api/v1/stock/entries.
* Owned by Inventory; Production and every other module post here — nothing writes the
* stock tables directly. Ledger-critical + race-safe inside the service.
```

The rule is realised as the `STOCK_POSTER` port. Six modules resolve that token and none of them touches a stock table: **sales** (dispatch), **purchase** (goods receipt), **production** (issue and output), **quality** (release and rejection), **maintenance** (spares consumption), and **inventory** itself (the implementation). The endpoint additionally *requires* an `Idempotency-Key` header and rejects the request with a field-level validation error if it is absent — a stock movement is the write you least want duplicated by a retried request.

3.4 The three chains XELOR actually runs

Chain	Path through the modules	Where the ledger is touched
Order to cash	Customer master → sales order + lines → credit gate → reservation → dispatch → GST invoice → receivable → trial balance	Stock out via <code>STOCK_POSTER</code> ; journal via <code>ACCOUNTS_POSTER</code> . Both synchronous.
Procure to pay	Vendor master → purchase order + lines → goods receipt → inspection gate → vendor invoice match → payable	Stock in via <code>STOCK_POSTER</code> after the <code>INSPECTION_GATE</code> clears.
Plan to produce	Item + BOM → MRP run → planned orders + pegging → capacity load → work order release → component issue → output → quality release	Issue and output both via <code>STOCK_POSTER</code> ; the work order never writes stock itself.

3.5 What XELOR deliberately does not do

- **It does not decide.** XELOR records what people and rules determine. Every recommendation in this product comes from ONYX and is labelled as such.
- **It does not hard-delete.** There is not a single `@Delete` route in the entire API (§8). Masters, financial and statutory rows are retired via `is_active`, never removed.
- **It does not reach outside the tenant.** No XELOR module opens an outbound connection to a customer's system; that is the connector shelf's job and it is unimplemented (§11.4).

VERIFY §3

```
grep -rl "STOCK_POSTER" apps/api/src/modules # the six consumers
grep -rc "@Delete(" apps/api/src # expect 0 across the tree
node -e "..." apps/web/src/modules/registry.ts # 24 registered modules
```

4. Data architecture

252 tables, 709 indexes, 96 migrations, one database. The conventions below are applied to every tenant-scoped table without exception, and three of them are checked mechanically.

4.1 Column conventions

Convention	Rule	Why it is not negotiable
Primary key	UUIDv7, generated in application code	Time-ordered, so index locality survives; no round trip to allocate; safe to generate before the transaction.
Tenant	<code>tenant_id</code> uuid not null on all 242 tenant-scoped tables; composite indexes lead with it	An index that does not lead with <code>tenant_id</code> is scanned across tenants before the policy filters it.
Provenance	<code>created_at/by</code> , <code>updated_at/by</code>	The audit trail explains a change; these say who last touched the row without a join.
Retirement	<code>is_active</code> soft delete. No hard <code>DELETE</code> on masters, financial or statutory rows.	MCA requires an eight-year audit trail. A deleted row cannot be audited.
Money	<code>NUMERIC(18,2)</code>	Float rounding in a tax computation is a compliance defect, not a display bug.
Rates and statutes	Effective-dated masters	A GST rate change must not rewrite last quarter's invoices.

4.2 Where the tables live

Domain	Tables	Notes
hrm	27	Largest single domain — employee, shift, roster, muster, leave, payroll and statutory outputs.
maintenance · expenditure	23 each	Asset lifecycle; budget and claim lifecycle.
csp	22	Ticket, SLA, complaint, warranty, AMC, spares.
planning	21	MRP run, planned order, pegging, capacity, exceptions.
administration · aiops · integration	19 · 17 · 16	RBAC and audit; AI governance; connectors and dead letters.
agent-os · quality · platform	11 · 9 · 9	Agent runs; inspection and NCR/CAPA; outbox, audit log, idempotency, event consumption.
fulfilment	7	ONYX's entire schema footprint — mission, step, plan version, action, approval, event, plus numbering.
accounts · sales · purchase · inventory	7 · 5 · 5 · 5	The financial and transactional core. Deliberately small; the complexity is in the rules, not the tables.
production · engineering · workflow · admin	3 each	Work order and components; item/BOM/BOM line; workflow instance and actions.
general · dataimport · numbering · copilot · platform-health	2 · 2 · 1 · 1 · 1	Company/plant; import batch and row; the document counter; copilot log; health checks.

4.3 The ten global tables

Ten tables carry no `tenant_id` and are therefore outside RLS. Each is deliberate and each was checked:

Table	Why it is global
<code>tenant</code>	The tenant registry itself. Cannot be tenant-scoped without circularity.
<code>_migrations</code>	Schema version ledger. Owned by the migrator role, not the application role.
<code>stat_epf_config</code> , <code>stat_esi_config</code> , <code>stat_gratuity_config</code> , <code>stat_ot_config</code> , <code>stat_pt_slab</code> , <code>stat_tds_config</code> , <code>stat_tds_slab</code> , <code>stat_wage_definition</code>	Indian statutory rate tables — EPF, ESI, gratuity, overtime, professional tax slabs, TDS and its slabs, and deemed-wage definitions. These are law, not tenant data : every tenant is subject to the same slab. Making them tenant-scoped would invite a tenant to edit its own tax rates. They are effective-dated instead.

This is the correct reading of a compliance requirement. "Every statutory number is config, never a constant in code" is satisfied by an effective-dated table; making that table tenant-writable would satisfy the letter and destroy the point. A reviewer should confirm the application role has no write grant on `stat_*` — `pnpm db:perm-check` covers grant cataloguing.

4.4 Migration discipline

96 forward-only SQL migrations, `0000_init.sql` through `0097_data_import_tenant_parent.sql`, applied by `pnpm db:migrate` as the owner role. The six that constitute ONYX are contiguous and identifiable:

Migration	What it adds
-----------	--------------

0092_one_active_bom_per_item.sql	Partial unique index guaranteeing a single active BOM revision per item — without it, the mission's BOM explosion double-counts.
0093_trace_spine.sql	The trace spine that links a mission step to the records it read and wrote.
0094_fulfilment_mission.sql	The six fulfilment tables and their RLS policies.
0095_fulfilment_numbering.sql	<i>MSN</i> - mission series registered in the platform counter.
0096_data_import.sql · 0097_data_import_tenant_parent.sql	Spreadsheet onboarding batch/row tables and their tenant parentage.

5. Multi-tenancy and row-level security

Tenancy is enforced in the database, by a role that cannot bypass it, on every one of the 242 tenant-scoped tables. This is the control a reviewer should test hardest, because every other isolation claim in the product rests on it.

5.1 The two-role model

Role	Used by	Capability
indcore_owner	Migrations only	Owns the schema. Table owners are exempt from RLS unless <code>FORCE</code> is set — which is exactly why <code>FORCE</code> is set on all 242.
app_user	The running API	NOBYPASSRLS , non-owner. Cannot see a row outside <code>app.current_tenant</code> even with a bug in the query. Bootstrapped by <code>infra/postgres/init/00-init.sql</code> .

The tenant is carried per-transaction in the `app.current_tenant` session GUC, set by `withTenant()`, which opens the transaction. Because the GUC is transaction-scoped, a pooled connection cannot leak a previous request’s tenant into the next one.

5.2 Coverage — measured, not asserted

VERIFY §5.2 — measured 16 Aug 2026 against `indcore_p2`

```
pnpm db:rls-check
RLS check OK — 242 tenant-scoped table(s) fenced
(FORCE RLS + a policy keying on app.current_tenant).

select count(*) from pg_tables where schemaname='public';                252
select count(distinct table_name) from information_schema.columns
  where table_schema='public' and column_name='tenant_id';                242
select count(*) from pg_class where relrowsecurity;                      242
select count(*) from pg_class where relforcerowsecurity;                 242
select count(*) from pg_policies where schemaname='public';              257
```

242 of 242 tenant-scoped tables have RLS **enabled and forced**. There is no partial coverage and no exemption list.

5.3 Two policy shapes, and the difference between them

The 257 policies are not identical. 248 use a `NULLIF`-guarded predicate; 9 do not.

```
-- the guarded form, 248 policies (e.g. sales_order)
tenant_id = (NULLIF(current_setting('app.current_tenant', true), ''))::uuid

-- the unguarded form, 9 policies (e.g. fulfilment_mission)
tenant_id = (current_setting('app.current_tenant', true))::uuid
```

The nine unguarded tables are exactly the newest ones: `fulfilment_mission`, `fulfilment_step`, `fulfilment_plan_version`, `fulfilment_action`, `fulfilment_approval`, `fulfilment_event`, `data_import_batch`, `data_import_row`, `quality_release`.

VERIFY §5.3 — the behavioural difference, measured

```
-- GUC absent entirely: both shapes fail closed, identically
select count(*) from sales_order;      0 rows
select count(*) from fulfilment_mission; 0 rows

-- GUC set to the empty string:
set app.current_tenant = '';
select count(*) from sales_order;      0 rows
select count(*) from fulfilment_mission; ERROR: invalid input syntax for type uuid: ""
```

Assessed accurately: this is a robustness defect, not an isolation defect. Neither shape leaks a row. Both fail closed when the tenant is unknown. The difference is that the guarded form returns an empty set and the unguarded form raises a database error, which surfaces as a 500 rather than a clean empty response and puts a Postgres error string into the log. It is worth correcting for uniformity and for log hygiene, and it is *not* a tenancy hole. Tracked as **R-2** in §18.

5.4 Isolation under test

A dedicated leak probe (`packages/db/src/rls/leak-probe.test.ts`) exercises cross-tenant reads against a second seeded tenant — **Kaveri ElectroFab Industries** — which exists in the demo universe for no other purpose than to be the tenant whose rows must never appear. The permission catalogue covers both tenants: 330 catalogue rows, 0 uncatalogued grants.

5.5 Performance of the fence

The binding decisions document states that RLS overhead above 15–20% triggers a mandatory design review. Measured on the current dataset:

Query as	Warm run 1	Warm run 2	Overhead
<code>indcore_owner</code> (RLS bypassed)	0.434 ms	0.460 ms	Inside measurement noise
<code>app_user</code> (FORCE RLS)	0.450 ms	0.415 ms	

This number does not generalise and must not be quoted as if it did. It was taken against 6,208 live rows in a 43 MB database — a demo dataset. At that volume the planner's work is dominated by fixed costs and the policy predicate is free. The 15–20% criterion is only meaningfully testable at production row counts on production-shaped indexes. **The overhead is currently unmeasured at scale**, and that is an open item (**R-4**, §18), not a passed test.

6. Transaction, consistency and eventing model

The rule that determines whether this system can be trusted with money is a single sentence: **ledger-critical writes stay synchronous, in one transaction, and never on the bus**. Everything else about the eventing design follows from wanting that to remain true.

6.1 What is synchronous, and what is not

Write	Mode	Reasoning
Stock movement	Synchronous	Posted through <code>POST /api/v1/stock/entries</code> inside one transaction, race-safe in the service. An eventually-consistent stock balance is a wrong stock balance.
Journal posting	Synchronous	Via <code>ACCOUNTS_POSTER</code> . A voucher that posts a second later is a voucher that can post in the wrong period.
Document numbering	Synchronous	Inside the caller's transaction (§7), so a failed insert cannot burn a number.
Audit entry	Synchronous	<code>audit.appendInTx(tx, ...)</code> — the chain link is written in the same transaction as the change it describes, or neither happens.
Cross-module notification	Asynchronous	Transactional outbox. The consumer's work is not part of the producer's correctness.
Report rendering, imports, long AI work	Asynchronous	BullMQ on Valkey.

6.2 The transactional outbox

`outbox_event` is written **in the same transaction as the domain change**, so an event can never describe a change that was rolled back, and a committed change can never fail to produce its event. A relay publishes and stamps `published_at`; `attempts` records redelivery.

```
outbox_event(id, tenant_id, name, payload, created_at, published_at, attempts)
  name is versioned: module.entity.verb.v1
  index ix_outbox_unpublished on (tenant_id, created_at)
```

Delivery is at-least-once. It is turned into an exactly-once *effect* on the consumer side by `event_consumption`, a dedup ledger with one row per `(consumer, event)` and a unique key that makes a redelivery a no-op. The guarantee is stated precisely in the schema comment:

```
// Consumer-side dedup ledger for the outbox relay (§5.4). One row per (consumer,
// event) actually handled; the unique key turns a redelivery into a no-op, so the
// bus's at-least-once guarantee becomes an exactly-once EFFECT. Append-only.
```

6.3 Request idempotency

Every mutating endpoint accepts an `Idempotency-Key` header; the stock endpoint *requires* one. The record is more careful than a bare key check:

Column	Purpose
<code>(tenant_id, key)</code> unique	Scopes the key to the tenant, so two tenants cannot collide.
<code>fingerprint char(64)</code>	SHA-256 of the request body. A replay with the <i>same</i> key and a <i>different</i> body is a client bug and is rejected, not silently answered from cache.
<code>status, response_status, response_body</code>	The saved answer, so a retried request returns the original response rather than re-executing.

ONYX derives its keys deterministically rather than randomly, e.g. `fulfil:{missionId}:v{versionNo}:procure:{vendorId}`. This is what makes a mission retry (§15) replay the documents already created and raise only the ones that were not — the vendor does not get the order twice.

6.4 The transaction-scoping rule engineers must know

`withTenant()` **opens a transaction and cannot be nested**. Any code path that reads through `withTenant`, then calls a port that opens its own transaction, then writes, must be written as three phases — read tx → port call (its own tx) → write tx. This is not a style preference; nesting produces a runtime failure. It is the single most common way to get an ONYX mission step wrong, because the ports are precisely the things that own their own transactions.

6.5 Where hard deletes exist — stated precisely

The API exposes **zero** `@Delete` **routes**. There are, however, four `tx.delete(...)` call sites in the codebase, and honesty about them matters more than the headline:

Location	Tables	Justification
<code>mission.service.ts</code> — retry	<code>fulfilment_step</code> , <code>fulfilment_approval</code>	A failed step row must be removed because the arc's resume point is a row count; a failed row would make the mission believe that step was done. The failure is written to the event log and the audit trail <i>before</i> the row goes, and the re-run opens with a <code>retrying</code> row carrying the original reason.
<code>mission.service.ts</code> — demo reset	All six <code>fulfilment_*</code>	Clears mission telemetry only, and refuses to run at all if the database holds any tenant outside the demo universe (<code>NOT_A_DEMO_DATABASE</code> , HTTP 403). It does not delete the purchase orders or work orders those missions created — they are real ERP documents.

No master, financial or statutory row is hard-deleted anywhere. The deletions are confined to ONYX's own execution telemetry.

7. Document numbering and statutory constraints

Fifteen call sites across six modules once built document numbers from a uuid fragment — so-358FA43E8CC9. It was unique and it sorted, so nothing failed. It was still wrong, and the reasoning behind replacing it is a good measure of how this codebase treats compliance.

Problem with a uuid fragment	Consequence
A tax invoice number is regulated	CGST Rule 46(b) requires a <i>consecutive</i> serial number, not exceeding sixteen characters, unique for a financial year. A uuid fragment is unique and sixteen characters and is not consecutive — so the series was not compliant. This is the one number in the system where correctness is a legal question rather than a matter of taste.
A document number is read aloud	A storekeeper quotes a GRN number over a telephone; a customer quotes an invoice number in an email. GRN-345C75F7FBFA cannot be dictated without spelling it, or checked by eye against a printed challan.
It carries no year	SO-2627-00014 states which financial year's series it belongs to. The uuid form says nothing, so a register cannot be reconciled by reading it.

7.1 How gaplessness is achieved

One counter row per (tenant, doc type, financial year). Allocation is a single UPDATE ... RETURNING, which takes a row lock held for the rest of the caller's transaction.

- Two clerks saving in the same millisecond **serialise on that row** and receive consecutive numbers.
- Because the increment is inside the caller's transaction, a document that fails to insert **does not burn a number** that would then be missing from the series for ever.
- A sequence would not give this.** PostgreSQL sequences are explicitly non-transactional; a rolled-back insert leaves a gap. Gaplessness is the requirement, so the slower design is the correct one.

Deliberate exceptions. CSP (csp_document_series) and EXPENDITURE keep their own equivalent counters rather than sharing the platform table. A reviewer should confirm those two implement the same in-transaction locking discipline — three counters that behave differently is a worse outcome than one that is slightly duplicated.

7.2 Other statutory facts held as configuration

Area	Treatment
GST	Effective-dated rate masters, including the 1 Aug 2026 Ship-to-GSTIN requirement. Two GSTINs are seeded in the demo universe (Pune-Chakan 27AABCT1234F1Z5, Coimbatore 33AABCT1234F1Z9) so that interstate/intrastate determination is genuinely exercised.
Payroll	EPF, ESI, gratuity, overtime, professional tax slabs, TDS and slabs, deemed-wage definitions — eight global effective-dated tables (§4.3).
Audit retention	MCA eight-year trail. Enforced by append-only, not by policy (§10).
Data protection	DPDP and CERT-In obligations recorded in the binding decisions document; PII egress control sits in the AI router's remit.

8. API contract

449 route decorators across 32 controllers, and the shape of every one of them is fixed by convention rather than by the taste of whoever wrote it.

234

POST routes

190

GET routes

25

PATCH routes

0

DELETE routes — by design

8.1 The canonical error envelope

```
{ "error": { "code": "VALIDATION_FAILED",          // stable, SCREAMING_SNAKE
  "message": "Request failed validation.",
  "details": [ { "field": "gstn",
    "message": "'BADGSTINSOY4FM9' is not a valid GSTIN" } ],
  "traceId": "efc753b2-aa71-4528-8427-..." } }
```

The `details` array is field-level and is carried all the way to the surface — a spreadsheet import that fails reports *which cell* and *why*, not "Request failed validation." `AppError` in `packages/platform/src/errors/error-envelope.ts` is the only thing that produces this shape, and common constructors (`Errors.validation`, `Errors.notFound`, ...) exist so that codes stay consistent across modules.

8.2 Pagination

Keyset cursors only. Offset pagination is rejected on two grounds recorded in `pagination.ts`: it *drifts under concurrent inserts* and it *scans linearly*. The cursor is an opaque encoding of `(createdAt ISO, id)`. Every list endpoint caps `limit` at **100**, e.g.

`Math.min(Number(limit ?? 25) || 25, 100).`

A trap for integrators, worth documenting for the reviewer. List envelopes are not uniform across the whole API — some collections return `{ items: [...] }` and others `{ data: [...] }`. Any client written against one shape will silently read `undefined` from the other. Tracked as **R-3** in §18.

8.3 Versioning and eventing names

HTTP is served under `/api/v1`. Events are versioned independently as `module.entity.verb.v1`, so an event contract can advance without a breaking HTTP change and vice versa.

8.4 Authorisation is declared on the route

Every route carries a `@RequirePermission("...")` decorator. The permission proof asserts that this is exhaustive in both directions:

VERIFY §8.4 — measured 16 Aug 2026

```
pnpm db:perm-check
routes demand 165 permission(s); registry declares 165.
catalogue covers 2 tenant(s); 330 catalogue row(s); 0 uncatalogued grant(s).
Permission check OK — 165 permissions; every route's demand is registered,
every registered permission is enforced, every grant is catalogued.
```

Both directions matter. "Every route's demand is registered" stops a route requiring a permission nobody can hold. "Every registered permission is enforced" stops a permission existing that no route consults — which is how a role appears to restrict something it does not.

9. Identity and authorisation

9.1 Authentication

Path	Mechanism	Notes
Normal	Keycloak 26 OIDC	Realm <code>indcore</code> , custom login theme under <code>infra/keycloak-themes/</code> . Token carries subject and tenant.
Public demo	Static header	<code>API_PUBLIC_DEMO=true</code> plus header <code>x-xelor-public-demo: investor-presentation</code> resolves to a fixed administrator context. Personas (<code>hari, ...</code>) select among fixed contexts; they are not an authentication mechanism and the code says so.

The blast radius of the demo bypass, stated plainly. When `API_PUBLIC_DEMO=true`, a static string in a header is a signed-in administrator. There is no secret, no expiry and no revocation. The controls that make this acceptable are: (a) it is off unless the environment variable is explicitly set; (b) `tenant.middleware.ts` pins the context to the two demo tenant ids; (c) the demo-reset endpoint independently refuses to run against a database containing any tenant outside that set. **The invariant a reviewer must hold the team to is that this flag never runs in front of real customer data.** It is currently guarded by convention and by the tenant pin — not by a mechanism that could not be misconfigured. Tracked as **R-1** in §18.

9.2 Authorisation

165 permissions, role-based, catalogued per tenant. 169 navigation permissions gate what the web application even offers. The web app holds a permission list; the API holds the enforcement. These are separate on purpose — the UI hiding a button is a courtesy, the API refusing the call is the control.

A real defect this separation caused, and what it teaches. A "New work order" button never rendered in public-demo mode because the hardcoded demo permission list omitted `production.order.create` — while the API allowed the call throughout. The failure was in the courtesy layer, not the control layer, which is the correct way round for such a bug to happen; but it demonstrates that the two lists can drift. The `module-check` gate now asserts all 169 nav permissions are platform-defined.

9.3 Segregation of duties

The administration module carries an explicit segregation-of-duties surface (`/admin`, 36 routes) rather than treating conflicting-role detection as a reporting afterthought. Approval authority in ONYX is separate again, expressed as autonomy tiers (§13.3).

10. Audit, evidence and tamper-evidence

The audit trail is not a log. It is a per-tenant hash chain protected by two independent mechanisms, and it is the artefact that makes every other claim in this product checkable after the fact.

10.1 The chain

```
entryHash = sha256( prevHash || "\n" || canonical(entry) )

canonical()  - deterministic JSON, object keys sorted recursively, so the hash is stable
seq          - per-tenant monotonic sequence: the chain order, independent of wall-clock
genesis      - seq 0 has no predecessor
```

Editing any historical row without recomputing every row after it breaks the chain, and a verify job re-walks it. `ai_action_log` chains identically through the same `chainHash` primitive — one implementation, because two chains eventually disagree.

10.2 Two independent enforcement mechanisms — both proved live

VERIFY §10.2 — executed against `indcore_p2`, 16 Aug 2026

```
-- 1. the application role has no grant at all
app_user> delete from audit_log where seq = 1;
ERROR:  permission denied for table audit_log

-- 2. the schema owner has the grant, and still meets the trigger
indcore_owner> delete from audit_log where seq = 1;
ERROR:  audit_log is append-only (MCA Rule 11(g)); DELETE is forbidden
CONTEXT:  PL/pgSQL function audit_log_append_only() line 6 at RAISE

indcore_owner> update audit_log set action = 'tampered' where seq = 1;
ERROR:  audit_log is append-only (MCA Rule 11(g)); UPDATE is forbidden
```

From `0000_init.sql`: a `BEFORE UPDATE OR DELETE` trigger raises, and — described in the migration as "belt-and-braces" — `REVOKE UPDATE, DELETE ON audit_log FROM app_user`. Defeating the audit trail requires defeating both, as the schema owner, from inside the database.

10.3 What is recorded

Column	Content
actor_id	Who. Never null — a system action carries the system actor, not an absence.
action	Dotted verb, e.g. <code>fulfilment.demo.reset</code> .
entity_type, entity_id	What was touched.
data jsonb	The field-level change set, produced by <code>change-set.ts</code> with deterministic stringification so the hash is reproducible.
prev_hash, hash	char(64) each — the tamper-evidence.

406 audit entries exist in the current demo database, written by ordinary operation rather than by a seeding routine.

11. ONYX — the intelligence layer (Phase 2)

ONYX adds 21 API files, 19 platform files, 6 migrations and 7 tables. It adds no table to any XELOR domain, no column to any XELOR table, and no import of any XELOR module. That constraint is the design, and it is what makes the layer portable to an ERP that is not XELOR.

11.1 The architectural rule

ONYX reaches XELOR only through injected port tokens. `apps/api/src/fulfilment/` must never import `modules/purchase` or `modules/production`. It resolves `PURCHASE_ORDER_WRITER` and `PRODUCTION_ORDER_WRITER` — Symbol tokens whose implementations are provided by the owning modules. The consequence is that the same intelligence layer can be pointed at a different implementation of those ports without touching a line of mission code, which is precisely the claim that ONYX can sit on somebody else's ERP.

11.2 The fifteen ports

Token	Declared in	Contract
STOCK_POSTER	<code>stock.port.ts</code>	The single stock write path. Six XELOR modules and ONYX all go through it.
ACCOUNTS_POSTER	<code>accounts.port.ts</code>	Sales invoice raising, journal posting, customer outstanding.
ITEM_PROVIDER, BOM_PROVIDER	<code>item.port.ts</code> , <code>bom.port.ts</code>	Item specs and released BOM structure.
INSPECTION_GATE	<code>inspection.port.ts</code>	Request inspection; read gate status. Goods receipt waits on it.
BIOMETRIC_DEVICE	<code>biometric.port.ts</code>	Attendance punches from a device.
WORKFLOW_EXECUTOR	<code>workflow.port.ts</code>	The workflow engine behind a port so the engine itself can be replaced.
DEMAND_SOURCE, STOCK_READER, BOM_GRAPH, PURCHASE_SUPPLY, PRODUCTION_SUPPLY, PRODUCTION_ORDER_CREATOR	<code>planning-inputs.port.ts</code>	The six read/write contracts MRP needs, so Planning depends on interfaces rather than on five sibling modules.
PURCHASE_ORDER_WRITER	<code>fulfilment-docs.port.ts</code>	ONYX → XELOR write. One PO per vendor, enforced by the input shape rather than by discipline. PURCHASE numbers and approves its own documents.
PRODUCTION_ORDER_WRITER		ONYX → XELOR write. PRODUCTION owns the release; the work order is pegged to the sales order line it was raised for.

11.3 Where ONYX gets its facts — nine connected, two stood in for, eight not connected

The source registry (`packages/platform/src/intelligence/sources.ts`) is served to the UI at `GET /fulfilment/sources`. It is unusually candid and that candour is load-bearing.

Status	Source	Tables read	Supplies
LIVE	Sales · Orders	<code>sales_order</code> , <code>sales_order_line</code>	The commitment: what, how many, at what rate, promised when. Written back via <code>reserved_qty</code> .
LIVE	Sales · Customers	<code>customer</code>	Who the promise is to — the name on every approval brief.
LIVE	Engineering · Items	<code>item</code>	Part numbers and UoM, so a shortage is a part and not a uuid.
LIVE	Engineering · BOM	<code>bom</code> , <code>bom_line</code>	Released structure. One active revision per item, so the explosion cannot double-count.
LIVE	Inventory · Stock	<code>stock_balance</code>	On-hand per item — the number the shortage is netted against.
LIVE	Purchase · Vendors	<code>vendor</code>	Which suppliers exist. A plan naming an absent vendor stops the mission.
LIVE	Purchase · Orders	<code>purchase_order</code> , <code>purchase_order_line</code>	Documents the mission raises — read back afterwards to prove they exist.
LIVE	Production · Work orders	<code>production_order</code>	The job on the shop-floor list, pegged to its sales order line.
LIVE	Planning · Runs & problems	<code>mrp_run</code> , <code>planned_order</code> , <code>plan_exception</code> , <code>plan_capacity_load</code>	Nothing, deliberately. The mission nets its own requirement per commitment; a plan run nets the whole factory. Recorded as an explicit non-integration rather than left to look like one.
STAND-IN	Supplier commercial terms	Seeded table <code>SEEDED_SOURCING</code>	Unit price, lead time, on-time reliability, capacity — the four numbers the strategy comparison turns on. Surfaced with provenance "seeded" on every step that uses it; overridable per mission by uploading a spreadsheet.
STAND-IN	Work-centre load	Constant <code>SEEDED_FACTORY.capacityHeadroom</code>	How much of the constraining work centre this batch can have. No MES or SCADA is connected to this build.

NOT CONNECTED **SAP** (ECC/S4 via OData) · **Tally** · **Odoo** (JSON-RPC) · **Dynamics 365** (Dataverse) · **MES/SCADA** · **Excel/CSV** · **REST API** · **read-only database replica** — eight connectors, every one `connected: false`, shown on screen because "ONYX can sit on your existing systems" is a real product claim, and carrying no green light because a demo implying a live SAP connection would make every true thing beside it worthless.

11.4 ONYX's own schema

Table	Holds
<code>fulfilment_mission</code>	One mission per sales order. Status, stage, autonomy tier, promised/forecast dates, confidence, margin, waiting reason, outcome.
<code>fulfilment_step</code>	One row per executed arc step: narration, evidence, findings, confidence, timing, pipeline. The resume point is a row count.
<code>fulfilment_plan_version</code>	Every plan considered, including superseded ones and <i>why</i> they were superseded.
<code>fulfilment_action</code>	Proposed → approved → executed → verified. Carries <code>digest</code> , <code>precondition</code> , <code>idempotency_key</code> , <code>autonomy_tier</code> , <code>approval_id</code> .
<code>fulfilment_approval</code>	The human decision, its brief, and the note the person wrote.
<code>fulfilment_event</code>	Everything that happened to the mission, with a CHECK-constrained <code>disposition</code> vocabulary.

12. The mission arc

A mission is thirteen named steps grouped into six chapters. The arc is declared once, as data, in `mission.service.ts`; the chapter of each step is derived from that declaration so the two cannot disagree.

#	Step	Chapter	Agent	Kind	Question it answers
1	intake	Understand the promise	ONYX	observe	What exactly has been promised, and by when?
2	engineering	Find out what is true	AXLE	observe	Is a released BOM in force for this product?
3	materials		SPAR	observe	What is short, after on-hand and inbound?
4	capacity		AXLE	observe	Can the floor absorb this batch on the promised date?
5	sourcing		SPAR	observe	Who is qualified to close the gap, and on what terms?
6	strategy	Choose a way through	ONYX	plan	Which feasible strategy best serves the commitment?
7	critique		HEXA	critique	Does the evidence support what this plan claims?
8	authorize	Ask, or proceed	HEXA	authorize	Is this inside the mission's authority?
9	reserve	Do it, and check it worked	SPAR	act	— reserves available stock against the order line
10	procure		SPAR	act	— raises the purchase orders through the port
11	workorder		KILN	act	— releases the work order through the port
12	watch	Prove the outcome	ONYX	wait	— dormant until something it depends on changes
13	close		HEXA	close	Was the commitment actually met?

12.1 The proposer never grades itself

Step 6 (ONYX) produces the plan; step 7 (HEXA) re-derives its numbers from the evidence independently. The engine interface makes this structural rather than procedural — `verify(chosen, evidence)` is a separate method with the comment *"The proposer never grades itself"*. Two of the planner's tests exist solely to prove the critic bites: *"the critic recomputes the date instead of believing the plan"* and *"the critic catches a plan that does not cover the shortage"*.

12.2 Waiting is a real state, not a pause

Step 12 puts the mission in status `waiting`, stage `monitoring`, and `advance()` refuses to move it:

```
// Waiting is a durable state, not a decorative step on the way to a fabricated
// closure. A supplier/production/quality/dispatch event must wake the mission; an SSE
// loop is not an event and may not advance it merely because the browser stayed open.
```

Step 13 similarly refuses to write a close row when the downstream gate is not satisfied — because writing a "waiting" close row would mark the step done, and resume is row-based. The mission parks and says why. In the demo world, all eight runnable scenarios terminate at step 12 in `waiting/monitoring`; this is correct behaviour and not an incomplete run.

12.3 Rejection is a real answer

A person facing the approval has three responses, all of which are terminal states the system handles: **approve**, **try another plan** (the refused strategy is scored out of contention and *stays in the list*, because deleting it would make the next plan look as though it never considered the obvious option), and **stop**. A rejection fails the mission and commits nothing — verified by counting purchase orders across a rejection: 75 before, 75 after.

13. ONYX's decision engine — determinism

The governing sentence is "AI explains, never decides." In this build it is stronger than that: for the fulfilment mission, AI does not even explain in the sense of generating text — the engine is deterministic arithmetic, and the only method a language model could own is isolated behind a single interface method that is not currently model-backed.

13.1 The engine interface

```
export interface IntelligenceEngine {
  readonly name: string;
  readonly kind: "deterministic" | "model"; // declared, and shown on screen

  assess(evidence) // what is true right now
  recommend(evidence, refused?) // compare the ways through and pick one
  verify(chosen, evidence) // re-derive. The proposer never grades itself.
  explain(request) // "This method — and only this one — a model could own."
  confidence(chosen) // how sure, and why
}
```

The shipped implementation is `DeterministicEngine`. No file under `packages/platform/src/intelligence/` contains an outbound HTTP call, an SDK import, or an API key reference. There is no network dependency in the decision path, which is why the demo runs fully offline and why the same evidence produces the same plan on any machine.

13.2 Determinism, asserted as a test

The planner's 19 behavioural tests are written as claims about the world rather than about the code. The last one is the guarantee itself:

Test	What it forbids
<i>the same evidence always produces the same plan, byte for byte</i>	Any nondeterminism — clock, randomness, map ordering, model sampling.
<i>SHORTEN THE RUNWAY AND THE PLAN CHANGES SUPPLIER — same code, different world</i>	A hardcoded answer masquerading as a decision.
<i>REMOVE THE SHORTAGE AND IT STOPS BUYING ALTOGETHER</i>	Buying as a default rather than a conclusion.
<i>DISQUALIFY THE CHEAP VENDOR AND IT IS NOT USED, however good the price</i>	Price overriding qualification.
<i>when nothing is feasible it escalates rather than picking the least bad</i>	A confident answer where there is no good one.
<i>every narrated sentence carries a checkable number</i>	Unfalsifiable narration.
<i>the comparison sentence names the runner-up — proof a choice was made</i>	Presenting one option as if it were a comparison.

13.3 Autonomy tiers — authority as configuration

Tier	Name	Expedite limit	Behaviour
A2	Suggest only	₹0	Plans and explains, then waits for a person before committing anything — including a stock reservation , because reserving inventory for one customer changes what every other order can promise.
A3	Act within limits	₹20,000	Takes routine reversible actions alone; asks before material commercial commitments.
A4	Act and notify	₹5,00,000	Commits up to ₹5 lakh of premium alone, and tells you afterwards.

A design detail worth noticing. A2 is not "A3 with a zero limit". The cheapest plan's premium is zero by definition, so a `premium > limit` test would wave it straight through — which is what happened before a separate `requireApprovalForAnyCommitment` flag was introduced. Two tests exist purely to keep that fixed: *"SUGGEST-ONLY asks even when the premium is zero — a zero limit could not express this"* and *"suggest-only asks before reserving stock even when there is nothing to buy"*.

13.4 And where a model is used, it is a different subsystem

The AI Operations module (ONYX, 33 routes, 17 tables) governs a provider-agnostic router with a closed feature registry, prompt lifecycle, cost ledger, PII egress control and a kill switch. Two CI gates apply:

- **The F1 gate** (`pnpm eval general.master_dedup`) grades the *detector* — does it find the duplicates.
- **The grounding gate** (`ai-grounding.ts`) grades the *explanation*. It exists because a local 3B model was measured **inventing an identifier and reversing a conclusion**. Its three rules: every conclusion must match the code-decided expectation, so the

model cannot move it; no note may contain a value absent from the evidence; no note may contradict its own headline. It exits 1 on any violation, so CI can block a provider that is not safe to ship.

This is the correct shape for AI governance. The code decides; the model is permitted to phrase; and a gate checks the phrasing against the evidence and fails the build. Both gates run in CI job 1 under `pnpm ai:eval`, against whatever `AI_PROVIDER` is configured — `stub` (offline) or `ollama` (local).

14. The AI module — providers, models and governance

This section answers the question a reviewer always asks and a demo rarely answers straight: **what, exactly, is the AI?** Is there a model? Whose? Running where? What happens when it is wrong, or slow, or off? The honest answer has three parts, and they are different from each other.

The three things called "AI" in this product, kept apart.

Name	Is it a model?	What it actually is
The ONYX decision engine §13, fulfilment missions	No. None.	Deterministic arithmetic and rules. No network call, no SDK, no key, no sampling. The same evidence produces the same plan byte for byte. <i>This is what drives the investor demo.</i>
The AI module this section	Optionally, locally	A provider-agnostic router in front of a closed registry — the canonical 8 features, plus one in evaluation and one explicit "no AI here" entry (§14.5). The bound provider is either an offline deterministic stub or a model running on the plant's own machine via Ollama.
ONYX the agent §12, mission arc	No	A label on 3 of the 13 arc steps, distinguishing the coordinating role from AXLE, SPAR, KILN and HEXA. An agent here is a named responsibility, not a process and not a model.

14.1 Is it connected through an API, an LLM, or Ollama?

Ollama, locally — or nothing at all. No hosted AI API is wired into this build. Exactly two providers implement the `AiProvider` interface, and `AI_PROVIDER` selects between them:

Provider	Model?	Network	Behaviour
stub StubProvider	None	None	Named <code>stub-deterministic</code> . Returns the shape a real model would return, computed by rules. Zero model spend , so the entire product — and every test — runs with no model present. This is the default in development and in the demo.
ollama OllamaProvider	Yes, local	HTTP to 127.0.0.1 only	Posts to <code>{OLLAMA_BASE_URL}/api/chat</code> , default <code>http://127.0.0.1:11434</code> . Default model <code>qwen2.5:3b</code> . <code>temperature: 0.1</code> , <code>stream: false</code> , 20-second <code>AbortSignal.timeout</code> . Falls back to the stub on any failure.

`apps/api/package.json` declares no hosted-AI SDK — no OpenAI, Anthropic, Azure, Bedrock or Vertex client is in the dependency tree, and there is no API-key environment variable for one. The only occurrences of those vendor names in the AI path are code comments naming them as future options, and one baseline identifier string (`azure_doc_intelligence_prebuilt_invoice`, §14.5). Across both providers there is exactly **one** `fetch()` call site in the entire AI module. Adding a hosted provider is deliberately a **one-line binding change in `AiModule`** — the router, the governance, the budget and the hash-chained log are identical whichever provider is bound.

14.2 Configuration surface

Variable	Default	Effect
AI_PROVIDER	stub	<code>stub</code> = offline and deterministic; <code>ollama</code> = local model.
OLLAMA_BASE_URL	<code>http://127.0.0.1:11434</code>	Loopback by default — nothing leaves the machine unless this is deliberately reprinted.
OLLAMA_MODEL	<code>qwen2.5:3b</code>	The <code>small</code> tier model.
OLLAMA_MODEL_PREMIUM	falls back to <code>OLLAMA_MODEL</code>	The HYBRID-tier lever: a larger local model for the <code>premium</code> tier.
OLLAMA_TIMEOUT_MS	<code>20000</code>	Beyond this the call is abandoned and the deterministic answer is used.
AI_DAILY_TOKEN_BUDGET	<code>2,000,000</code>	Per-tenant daily ceiling; a per-tenant override lives in the ledger.

14.3 The router — one doorway, four steps, in order

Every AI call in the product passes through `AiRouterService.complete()`. It is thin by design and contains no business logic — "what a result MEANS is the owning module's job".



A refusal is itself an auditable AI action: it is written to `ai_action_log` with `degraded: true` and the reason, *then* the call fails closed with `AI_REFUSED` (403) so the caller drops to the feature's deterministic degraded mode. An unregistered feature key throws `AI_FEATURE_NOT_REGISTERED` before any token is spent.

14.4 The PromptPack contract — the model phrases, it never concludes

A provider does not know how to prompt for a feature; the feature contributes that, and — critically — contributes the validator too.

```
export interface PromptPack<I, O> {
  system: string;           // "Keep it narrow — the model phrases, it never concludes."
  buildUser(input: I): string;
  parse(text: string, input: I): O;
  // MUST throw if the text is ungrounded or malformed —
  // the provider then degrades to the deterministic answer.
}
```

Three consequences follow, and they are the whole safety story:

- **A feature with no PromptPack never reaches a model at all.** `OllamaProvider` returns the deterministic answer marked `degraded: "no_prompt_pack"` — so the log shows AI did not really run. A feature is opted *in* to live inference, never opted out.
- **An ungrounded reply is discarded, not shown.** `parse()` throwing means "the model said something we will not show a user", and the call falls through to the deterministic answer.
- **The ERP never blocks on a model.** Unreachable, slow, HTTP error, malformed reply — every path degrades. There is no code path in which a model being down stops a factory from working.

14.5 The closed feature registry

AI is not a capability that can appear anywhere someone adds a call. It is a closed list of ten entries — the canonical eight, one feature explicitly in evaluation, and one explicit *null* entry recording that Integrations has no MVP AI. Anything not on this list is rejected by the router.

Ref	Feature	Owner	Status	Risk	Data class	Deterministic baseline it must beat	Degraded mode
AI #1	Receipt / document extraction + auto-categorisation	expenditure	committed	2	pii_minimised	azure_doc_intelligence_prebuilt_invoice	manual_entry
AI #2	Master-data duplicate detection	general	committed	2	business	pg_trgm_gstin_exact	deterministic_substitute
AI #3	Ticket auto-triage + sentiment	csp	committed	1	pii_minimised	keyword_rule_classifier	feature_hidden
AI #4	Duplicate-receipt detection	expenditure	stretch	1	business	attachment_sha256_exact	deterministic_substitute
AI #5	HSN / SAC code suggestion	general	stretch	2	business	tax_code_directory_lookup	deterministic_substitute
AI #6	Reply drafting / thread summarisation	csp	stretch	2	pii_minimised	canned_response_template	feature_hidden
AI #7	Payslip explainer	hrm	stretch	3	pii_minimised	payslip_trace_template	template_output
AI #8	Segregation-of-duties conflict explanation	administration	stretch	3	business	sod_rule_static_sentence	template_output
AI #9	Copilot — read-only Q&A over the tenant's own data	copilot	in_eval	3	business	keyword_intent_router	deterministic_template
—	(explicit null entry — no MVP AI)	integrations	no_mvp_ai	—	none	deterministic_dlq_triage_table	no_mvp_ai

Two registry fields are doing unusual and valuable work. `deterministicBaseline` names the rules-only comparator the eval gate must *beat* — so a feature cannot ship on the grounds that it is "AI", only on the grounds that it outperforms the boring alternative. And `degradedMode` is declared per feature at design time, which is why "the model is down" has a defined product behaviour for every one of the ten rather than an exception.

14.6 Governance — fail-closed, ordered, and audited

Before a single token is spent, `DbAiGovernance.check()` asks three questions in a fixed order and refuses on the first failure:

Order	Gate	Meaning
1	Tenant opt-out (DPDP)	A tenant that has opted out of AI processing gets <code>opt_out</code> . This is checked first because consent precedes everything else.
2	Kill switch	Per feature, or tenant-wide via <code>'*'</code> . Two control surfaces converge here — <code>ai_kill_switch</code> (the AI Operations console) and <code>ai_feature_state</code> . They were unified after the console could display an engaged switch while real model calls continued.
3	Daily token budget	Per tenant per day, default 2,000,000 tokens. Exhaustion refuses rather than overspending.

Every administrative action — engaging or releasing a kill switch, setting an opt-out, changing a budget — **requires a reason** and appends a hash-chained audit row, the same posture Administration takes everywhere else (§10).

`ai_action_log` records every call: feature key, input, output, model, tier, token usage, and the `degraded` flag. It is chained with the identical `chainHash` primitive as `audit_log` (§10.1) — one implementation, because two chains eventually disagree. **A refusal is logged as carefully as a success**, so "the AI did not answer" is as provable as what it said.

14.7 The two CI gates

Gate	Grades	Rule
<code>pnpm eval general.master_dedup</code>	The detector	Does it find the duplicates? F1 against a golden set, and it must beat the <code>deterministicBaseline</code> .
<code>ai-grounding.ts</code>	The explanation	(1) Every conclusion must match the code-decided expectation — the model cannot move it . (2) No note may contain a value that is not in the evidence. (3) No note may contradict its own headline. Exits 1 on any violation.

Why the grounding gate exists — a measured failure, not a hypothetical. A local 3B model was observed **inventing an identifier and reversing a conclusion** on the master-dedup explanation. The response was not a better prompt; it was a gate that fails the build. Both gates run in CI job 1 under `pnpm ai:eval`, against whatever `AI_PROVIDER` is configured — so a provider that is not safe to ship cannot be shipped.

14.8 Why local-first — the benefits, and what each one buys

Benefit	Mechanism	What it is worth
Data residency	Loopback base URL by default. Nothing leaves the site.	DPDP and CERT-In obligations are met by <i>topology</i> , not by a processing agreement. For an Indian MSME whose costing and customer data is its business, this is often the whole objection answered.
Zero marginal cost	No per-call billing; the stub costs nothing at all and Ollama costs electricity.	Unit economics do not degrade with usage, so AI can be on by default rather than rationed. It also makes 995 tests runnable without a spend budget.
Works offline	The demo and the entire test suite run with no model and no internet.	A factory in Coimbatore with an unreliable link is a target customer, not an edge case. It also means an investor demo cannot fail because of conference wi-fi.
No vendor lock-in	Provider-agnostic router; swapping is a binding change in <code>AiModule</code> .	Model pricing and capability are moving quarterly. The architecture treats the provider as replaceable rather than foundational.
Never blocks operations	Every feature declares a <code>degradedMode</code> ; every failure path degrades.	The ERP's availability is independent of the model's. A model outage is a quality reduction, never an outage.
Provable behaviour	Hash-chained <code>ai_action_log</code> including refusals; kill switch; per-tenant budget.	"What did the AI do on 14 March, and who authorised it" is answerable — which is what a regulated or audited customer will eventually ask.
Honest evaluation	Every feature must beat a named rules-only baseline.	Stops the most common failure in AI products: shipping a model where an index would have been better, and calling it progress.
Tiering is demonstrable	CLOUD / HYBRID / EDGE, with <code>OLLAMA_MODEL_PREMIUM</code> as the HYBRID lever.	The deployment story is a configuration change rather than a rewrite — "aspirational" becomes "already running".

14.9 The costs of this choice, stated

- **A 3B local model is materially weaker** than a frontier hosted model. The grounding gate exists precisely because of what that weakness produced. Features whose value depends on strong reasoning are marked `stretch`, not `committed`.
- **Local inference needs local hardware.** The per-plant hardware requirement, and who pays for it, is a commercial question this document does not settle.
- **No hosted provider has been exercised end to end.** The router is provider-agnostic by construction and by interface, but that claim has not been proved by actually binding a hosted provider and running the gates against it. Tracked as **R-11** in §18.
- **The AI module is not what the fulfilment demo runs on.** Missions are decided by the deterministic engine (§13). A reviewer impressed by the mission arc should not attribute it to a model, and this document should not let them.

VERIFY §14

```
grep -rn "openai\\|anthropic\\|azure\\|bedrock\\|vertex" apps/api/src/ai packages/platform/src/ai
# no hosted-provider client or key in the AI path
grep -rn "fetch(" apps/api/src/ai/*.provider.ts
# exactly one call site: ${OLLAMA_BASE_URL}/api/chat
grep -rn "fetch(\\|http" packages/platform/src/intelligence/*.ts
# none — the ONYX decision engine has no network dependency at all
pnpm ai:eval          # F1 gate + grounding gate, both must exit 0
```

15. Failure, retry and recovery

15.1 A failed step stops the mission and says so

Scenario 8 exists to demonstrate this and is the only demo scenario designed to fail. Its recorded reason is written to be indistinguishable in shape from a real refusal:

```
status: failed · stage: execution · steps: 10
waitingReason: "purchase order creation/verification failed – Simulated fault, armed by
demo scenario 8 – PURCHASE was never called. A real refusal here would read exactly like
this: the vendor is on a credit hold that only a person can lift."
```

15.2 Retry replays rather than duplicates

POST /fulfilment/missions/:id/retry re-runs the last failed step. It is safe for a stated reason: every execute step derives its idempotency key from the mission, the plan version and the vendor — and from nothing that varies between attempts. So a re-run replays the documents that already exist and raises only the ones that do not. The vendor does not get the order twice.

And the failure is not swallowed. The failed step row must be removed, because the arc's resume point is a row count and a failed row would make the mission think that step was done. Before it goes, the failure is written to the event log and the hash-chained audit trail, and the re-run opens with a `retrying` row carrying the reason the first attempt failed. The code comment states the stakes exactly: *"A retry that quietly produced a clean-looking mission would be the most dangerous feature in this product."*

15.3 Concurrency and replanning

Condition	Handling
Two advances race	<code>isConcurrentAttempt(e)</code> collapses the loser into the winner's status rather than erroring.
A second mission on one order	<code>start</code> is idempotent — it returns the live mission rather than opening a second. The demo's own narration says "was already running, N steps in" rather than "opened", because claiming otherwise would be the first inaccurate sentence of the demo.
The world changes mid-mission	Status <code>replanning</code> , a new plan version, and the superseded version retained with its reason.
A person refuses the approach	Marked <code>operator refused the strategy</code> — distinguished from "the world changed", because a supplier delay invalidates a <i>plan</i> whereas this invalidates an <i>approach</i> , and the approach must not be proposed again.
Downstream not yet proven	Mission parks in <code>waiting</code> with <code>outcome not yet proven – {reasons}</code> . No close row is written.

15.4 Integration failures

The integration module (32 routes, 16 tables) carries a dead-letter surface with replay. Spreadsheet import failures are reported per row with the field and the value that failed, and a batch commits row by row through the domain service rather than by bulk insert — so a bad row cannot take a good one with it.

16. Verification evidence

Everything in this section was executed on 16 August 2026 against the checked-out source and the running stack. Commands are given so a reviewer can disagree with the numbers by producing different ones.

16.1 Automated tests

Suite	Tests	Pass	Fail	Skip	Character
@ind-core/platform	822	822	0	0	Pure logic — tax, planning, quality, accounting, audit, fulfilment, intelligence. No database.
@ind-core/api	104	103	0	1	Service and correctness tests. The skip is environment-gated.
@ind-core/web	58	58	0	0	Spine logic — pipeline parsing, mission arc, permissions.
@ind-core/db	6	6	0	0	Schema integrity, RLS leak probe, data-import integrity.
@ind-core/edge	6	6	0	0	Serverless runtime.
Total	995	994	0	1	0 failures across every suite.
Playwright e2e	39	27	0	12	Browser. The 12 skips need a Keycloak sign-in that public-demo mode bypasses — see R-5.

16.2 The five structural proof gates

Gate	Result, verbatim
db:rls-check	PASS RLS check OK — 242 tenant-scoped table(s) fenced (FORCE RLS + a policy keying on <code>app.current_tenant</code>).
db:perm-check	PASS routes demand 165 permission(s); registry declares 165. Catalogue covers 2 tenant(s); 330 catalogue row(s); 0 uncatalogued grant(s).
db:naming-check	PASS 252 tables; <code>production_order</code> and <code>maintenance_work_order</code> are distinct doctypes with no FK between them.
module-check	PASS 24 module(s) installed and registered; 169 nav permission(s) all defined by the platform; no cross-module imports .
lint / typecheck	PASS <code>eslint --max-warnings=0</code> including the boundary rules; <code>tsc -b</code> clean across all five packages.

16.3 Functional evidence — the nine demo scenarios

Each scenario resets the demo world, opens a mission on an order the resolver proves is suitable, and runs the arc.

#	Scenario	Result	Steps	What it establishes
1	stock-covers-it	waiting/monitoring	12	"Nothing needs buying" is a conclusion, not a default. The procure step says so rather than skipping quietly.
2	short-of-material	waiting/monitoring	12	Netting against live stock; 2 of 5 components short.
3	recommend-purchase	waiting/monitoring	12	Raises real purchase orders through the port.
4	recommend-work-order	waiting/monitoring	12	Releases a real work order pegged to the sales order line.
5	missing-information	BLOCKED	—	Cannot start: the seed contains no order lacking a released BOM or a qualified supplier. See R-6.
6	human-approval	waiting/monitoring	12	Stops at step 8 for a signature. 1 approval consumed.
7	spreadsheet-source	waiting/monitoring	12	A price list arriving as a spreadsheet overrides the stand-in terms for one mission.
8	failure-then-retry	failed (by design)	10	Fails at the purchase step, names the reason, and is retryable.
9	full-audit-trail	waiting/monitoring	12	End to end with every action verified.

16.4 The claim that matters most, proved at database level

VERIFY §16.4 — ONYX writes real XELOR documents

```
POST /fulfilment/demo/reset          POs=13  WOs=13  missions=0
<run scenario 3, recommend-purchase> POs=14  WOs=14  missions=1

select po_no, status, total_amount from purchase_order order by created_at desc limit 1;
      PO-2627-00014 | draft | 4457.50

select action_type, status, verified, title from fulfilment_action;
inventory.reserve | verified | t | Reserve stock against SO-2627-00002
purchase.commit   | verified | t | Raise 1 purchase order(s) for SO-2627-00002
production.release| verified | t | Release 1 work order(s) for SO-2627-00002
```

All three actions reach `verified` — the mission re-reads what it wrote and confirms it exists. A governance counter-proof was also run: a rejected approval fails the mission and commits nothing, with the purchase order count unchanged across the rejection (75 → 75).

16.5 Measured performance — and its limits

Operation	Measured	Conditions
GET /sales/orders?limit=25	11–22 ms	6,208 live rows · 43 MB database · single local machine. These are demo-scale figures.
GET /purchase/orders?limit=25	12–15 ms	
GET /inventory/stock?limit=25	10–14 ms	
GET /fulfilment/scenarios	49–87 ms	
Full 12-step mission	587 ms	12 sequential HTTP advances plus polling reads.

Two honest caveats. (1) `GET /fulfilment/scenarios` is 4–6× the other endpoints because it explodes the BOM and nets stock for every confirmed order to decide which scenarios can run — a cost the code acknowledges and caps. It is a demo-support endpoint, not a hot path. (2) None of these figures constitute a performance test. There has been no load test, no concurrency test, and no measurement at production row counts. See **R-4**.

17. Build, run and deploy

17.1 Local

```
# first time
docker compose -f infra/docker-compose.yml up -d
psql "$DATABASE_OWNER_URL" -v ON_ERROR_STOP=1 -f infra/postgres/init/00-init.sql # two-role model
pnpm install --frozen-lockfile && pnpm build && pnpm db:migrate
pnpm demo:seed && pnpm demo:northstar

# run one phase, and only one
./run-phase.sh 1 # PHASE-1 api :3000 web :3001 db indcore
./run-phase.sh 2 # PHASE-2 api :3100 web :3101 db indcore_p2
./run-phase.sh stop

# before any presentation
pnpm demo:rebuild && pnpm demo:verify
```

The failure mode to know. The web app proxies `/api/v1` to `NEXT_PUBLIC_API_ORIGIN` and defaults to `:3000`. For ONYX that default is *XELOR's* API — which answers, and has no `/fulfilment` routes, so the browser gets a clean 401 and the screen reports a technical error while every direct curl against `:3100` passes. `run-phase.sh` sets the variable correctly; a hand-started process may not.

17.2 Continuous integration

Job	Gates
1. Static and schema proofs	Bootstrap two-role DB → build → lint (boundary rules) → typecheck → migrate → unit tests → ai:eval (F1 + grounding) → rls-check → perm-check → naming-check → module-check → report arithmetic proof → deployment descriptor syntax.
2. Clean image builds	Five images from a clean context: <code>standard-api-worker</code> , <code>standard-api-migrator</code> , <code>standard-web</code> , <code>railway-api</code> , <code>railway-web</code> . Building from a clean context is what catches a Dockerfile that only works because of a developer's local files.
3. Demo rebuild and acceptance	Migrate → seed → start API → start web → <code>pnpm demo:verify</code> acceptance matrix → browser smoke test.

17.3 Deployment surface

Artefact	Role
infra/docker-compose.yml	Local dependencies: postgres, valkey, keycloak, gotenberg.
infra/docker-compose.prod.yml	Production composition.
infra/Caddyfile	Single public surface; TLS termination and routing.
infra/railway/	Managed-platform descriptors, syntax-checked in CI.
infra/postgres/init/00-init.sql	Creates <code>indcore_owner/app_user</code> , the three required extensions and the grants. Skipping this is the most common cause of a failed first migration (<code>ResolveOpClass</code> at the <code>pgvector</code> step).
infra/keycloak-themes/indcore/	Branded login theme.

Target region is AWS `ap-south-1` for data-residency reasons, provisioned with OpenTofu, observed with OpenTelemetry, Grafana and Sentry.

18. Gap, risk and decision register

Eleven open items. Each states the condition, the actual consequence, and what would close it. Nothing here is hypothetical — every item was observed on this build.

ID	Severity	Condition	Consequence	What closes it
R-1	HIGH	Public-demo bypass: a static header is a signed-in administrator when <code>API_PUBLIC_DEMO=true</code> .	No secret, no expiry, no revocation. Mitigated by the tenant pin to the two demo ids and by <code>NOT_A_DEMO_DATABASE</code> on reset — but the flag reaching a real-data environment is a configuration mistake away.	Make the guard structural: refuse to boot with <code>API_PUBLIC_DEMO=true</code> if the database contains any tenant outside the demo set, rather than checking only at reset time.
R-2	MED	9 of 257 RLS policies omit the <code>NULLIF</code> guard (all six <code>fulfilment_*</code> , both <code>data_import_*</code> , <code>quality_release</code>).	Not an isolation defect — all fail closed. With the GUC set to empty string they raise <code>invalid input syntax for type uuid: ""</code> instead of returning zero rows, surfacing as a 500 and putting a Postgres error in the log.	One migration rewriting the nine predicates to the guarded form; extend <code>rls-check</code> to assert policy <i>shape</i> , not just presence.
R-3	MED	List envelopes are not uniform: some endpoints return <code>{items: [...]}</code> , others <code>{data:[...]}</code> .	Any client written against one shape silently reads <code>undefined</code> from the other. Cost several hours of integration debugging during this review alone.	Pick one, migrate the minority, and add a contract test that fails on a list response using the other key.
R-4	HIGH	No performance testing at scale. RLS overhead measured only at 6,208 rows.	The binding decision "RLS overhead >15–20% triggers a design review" is currently untestable — the criterion exists but has never been evaluated under conditions where it could fail. Composite-index-leads-with-tenant_id discipline is likewise unvalidated at volume.	A seeded volume dataset (≥10 ⁶ rows on the hot tables), a load profile, and the RLS comparison re-run against it.
R-5	MED	12 of 39 e2e tests are environment-skipped in the public-demo configuration.	Sign-in, persona isolation and credential rejection are never exercised in the configuration that is most often run. A regression in the Keycloak path would not be caught by a green suite.	A second CI matrix leg that runs the suite with Keycloak enabled and <code>API_PUBLIC_DEMO=false</code> .
R-6	LOW	Demo scenario 5 (missing-information) cannot start.	The resolver honestly reports that no seeded order lacks a released BOM or a qualified supplier, so 1 of 9 scenarios shows UNAVAILABLE in the picker. The refusal behaviour it demonstrates is therefore never shown.	Seed one confirmed order whose finished good has no active BOM, or one shortage with no qualified supplier.
R-7	LOW	<code>demo/reset</code> clears missions but retains the purchase and work orders they raised.	Deliberate — they are real ERP documents and "a demo convenience is not a reason to make them erasable". But document numbers climb and stock depletes across repeated reruns, which can eventually flip scenarios to UNAVAILABLE mid-presentation.	Presenter guidance, or a separate full-world rebuild (<code>pnpm demo:rebuild</code>) between sessions.
R-8	LOW	The mission's step-by-step pipeline visual is present on 4 of the 10 module screens the tour visits, plus Mission Control.	On the other 6 screens the Trigger→Collect→...→Record breakdown is not shown, so the "show the pipeline everywhere" requirement is partially met.	Mount the stage panel on the remaining screens, or return the rail to the shell-level copilot.
R-9	LOW	The copilot card attributes every step to ONYX.	The arc assigns steps to AXLE, SPAR, KILN and HEXA (§12), so the card's "ONYX used..." sentence disagrees with the step's own <code>agentKey</code> on 8 of 13 steps.	Render <code>current.agentKey</code> and its role rather than a constant.
R-11	MED	Provider-agnosticism of the AI router is proved by interface, not by exercise.	No hosted provider has ever been bound and run through the router, the governance gates, the budget ledger and the two CI eval gates. The abstraction is clean and the seam is one line — but "it will swap cleanly" is currently a design claim, and token accounting, latency behaviour and error mapping for a hosted provider are all untested.	Bind one hosted provider behind a feature flag, run <code>pnpm ai:eval</code> against it, and keep it as a second CI matrix leg.
R-10	MED	Mobile/offline-first strategy is undecided.	Named in the governance document as the largest open gap and a blocking gate before the UX freeze and the manufacturing auth pack. A shop floor is the one place a browser session cannot be assumed.	An ADR reviewed by HEXA, before the UX freeze rather than after.

18.1 Defects found and fixed during this review

Recorded because they indicate what the verification process catches, and how.

Defect	How it was found	Fix
The approval card named screens that do not exist — "Purchase → Orders" where the product's screen is "Purchase orders".	Comparing <code>applicationTargets</code> in <code>narrate.ts</code> against the step registry's <code>where</code> labels in <code>mission.service.ts</code> .	Names matched to the registry; asserted literally in <code>planner.test.ts</code> .

The centred copilot card (z-120) covered 40% of the application's own modal (z-50) and made its centre unclickable.	Live measurement with <code>elementFromPoint</code> at the dialog's centre, which returned the card.	The card yields to any <code>aria-modal</code> dialog; e2e regression test added.
ONYX's entire e2e suite defaulted to <code>localhost:3001</code> — XELOR's web server.	Repointing the config surfaced two specs with hardcoded origins.	Config and all 8 specs repointed to <code>:3101/:3100</code> .

19. The reviewer's checklist

If you have one hour, these are the twelve questions worth asking, with where the answer is and what a wrong answer looks like.

#	Question	Where the answer is	A wrong answer looks like
1	Is tenant isolation enforced by the database, or by the application?	§5. <code>app_user</code> is NOBYPASSRLS; 242/242 tables FORCE RLS.	"The service always adds <code>where tenant_id.</code> " One missed query is then a breach.
2	Can the module boundary actually be violated?	§2. <code>eslint-plugin-boundaries</code> , <code>--max-warnings=0</code> , CI job 1.	A README rule; or the TS resolver quietly unconfigured, so the gate classifies nothing.
3	What happens to the ledger if the queue is down?	§6.1. Stock, journal, numbering and audit are all synchronous.	"The event handler posts the journal."
4	Can the audit trail be edited by anyone?	§10.2. Trigger <i>and</i> REVOKE, both proved live against both roles.	A single mechanism; or an application-level "don't update this" convention.
5	Does the AI decide anything?	§13. Deterministic engine, no network in the decision path, <code>verify()</code> separate from <code>recommend()</code> , autonomy tiers gating commitment.	A model in the decision path; or an "explain" that can move the conclusion.
6	What is the AI — a hosted model, a local one, or none?	§14.1. Two providers: an offline deterministic stub, or a local model via Ollama on <code>127.0.0.1</code> . No hosted SDK in the dependency tree. The fulfilment demo uses no model at all (§14, first table).	"It uses AI" without naming the provider, the model, where it runs, or what happens when it is down.
7	Is ONYX's separation from XELOR real or presentational?	§11.1–11.2. 15 Symbol tokens; zero module imports; 7 tables all its own.	A direct import of <code>modules/purchase</code> from the fulfilment tree.
8	What stops a retried request from double-posting?	§6.3. <code>(tenant, key)</code> unique + SHA-256 body fingerprint + saved response; deterministic keys in ONYX.	A key check with no body fingerprint — a different body under the same key returns the wrong cached answer.
9	Are invoice numbers legally issuable?	§7. Gapless per <code>(tenant, doctype, FY)</code> via in-transaction <code>UPDATE ... RETURNING</code> .	A Postgres sequence — non-transactional, so a rolled-back insert leaves a gap.
10	What is genuinely connected, and what is staged?	§11.3. 9 live, 2 stand-ins labelled at every surface, 8 connectors <code>connected: false</code> .	A connector list with green lights and no runtime behind them.
11	Does it perform?	§16.5 and R-4 . Unknown at scale . Demo-volume figures only.	Quoting the 10–22 ms figures as though they were a load test result.
12	What is the weakest link today?	R-1 (demo auth bypass) and R-4 (no scale testing), in that order.	An answer that names no weakest link.

19.1 The summary a reviewer can rely on

What this build genuinely demonstrates

- Database-enforced multi-tenancy on 242/242 tables, with a non-bypassing application role.
- Mechanically enforced module boundaries — 24 modules, zero cross-module imports, checked in CI.
- A tamper-evident audit chain protected by two independent mechanisms, both demonstrable.
- An intelligence layer that reaches the ERP only through 15 declared ports and writes real, verifiable documents.
- A deterministic decision engine with no network dependency, an independent critic, and authority expressed as configuration.
- An AI module that is honest about itself: a closed 10-entry registry, local-or-no model, fail-closed governance, a hash-chained call log, and every feature required to beat a named rules-only baseline.
- 995 automated tests and five structural proof gates, all currently green.

What it does not yet demonstrate

- Any behaviour at production scale — no load test, no concurrency test, RLS overhead unmeasured where it could fail.
- A safe authentication story for the demo configuration (**R-1**).
- A single live connector to any external ERP — all eight are declared and none is implemented.
- Sign-in and persona isolation under CI, which the current configuration skips (**R-5**).
- Mobile or offline operation, which is undecided (**R-10**).
- A hosted AI provider exercised end to end — the router is provider-agnostic by construction but has only ever run a local model or the offline stub (**R-11**).
- Autonomous external execution. The product does not claim it, and the code does not do it.

The single sentence to take away. XELOR's architecture is stronger than its operational evidence: the structural controls — tenancy, boundaries, audit, ports, determinism — are enforced by machines rather than asserted by documents, and each was re-proved while writing this dossier. What is missing is not design; it is scale evidence and the closing of a small, named set of gaps, every one of which is listed in §18 with the work that would close it.

XELOR Platform Architecture & Verification Dossier · 16 August 2026 · Measured against `PHASE-1` and `PHASE-2` at PostgreSQL 17.10, 96 migrations, 995 automated tests.

Supersedes the *MVP Technical Handoff Brief* (8 August 2026), which covered the XELOR (Phase 1) investor build only. Every figure here is reproducible with the commands given in each `VERIFY` block.